Smart Campus Energy Consumption Tracker & Analyzer

Project report | PIYUSH TAK | Synthetic demonstration

Abstract

This project implements a nine-stage campus electricity analysis workflow using Python, Pandas, Matplotlib, Scikit-learn and Streamlit. It combines hourly data-quality checks, exploratory analysis, Random Forest forecasting, Isolation Forest investigation alerts and conditional cost-saving scenarios. No actual campus meter readings were provided.

Nine-stage workflow

1. Problem definition: Identify peak usage and investigation priorities.

2. Dataset creation: 120 days; 5 buildings; 14,400 synthetic hourly rows.

3. Data collection: Timestamp, building, interval kWh, occupancy, appliances, temperature, holiday.

4. Cleaning: Invalid readings, duplicates, missing-hour checks, past-only limited fills.

5. EDA: Daily/hourly patterns, buildings, weekends, temperature.

6. Features: Calendar, building, lag 1/24/168, shifted rolling mean.

7. Modeling: Random Forest, previous-day baseline, Isolation Forest.

8. Visualization: Actual vs predicted and anomaly markers.

9. Dashboard: CSV upload, forecast, alerts, scenario savings, exports.

Dataset and cleaning

The seeded synthetic dataset contains Academic Block, Library, Hostel, Laboratory and Administration. It includes daily/weekly patterns, temperature associations and injected unusual usage. Seventy missing energy observations are intentionally introduced. Energy is hourly interval kWh, not a cumulative meter register. Missing energy is forward-filled at most three hours for feature continuity; imputed targets are excluded from fitting and evaluation. Totals display observed energy only.

Modeling and evaluation design

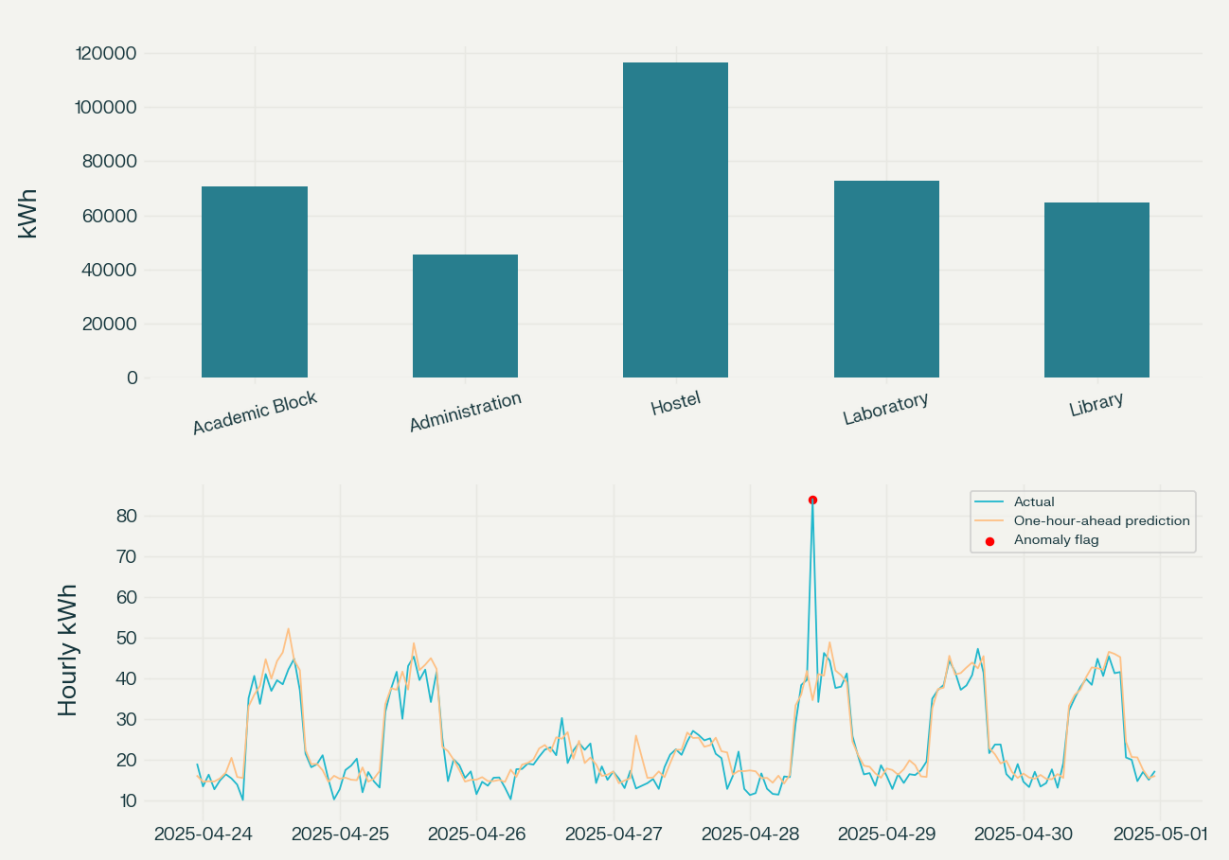
A common chronological cutoff uses the earliest 80% of usable timestamps for training and the remaining 20% for evaluation. All lags and rolling means use past energy only. Earlier actual holdout readings become available for later one-hour-ahead predictions. The baseline is energy from 24 hours earlier. A separate production forest is refitted on all usable historical targets for recursive future forecasts. Future occupancy and temperature are not predictive inputs.

Executed synthetic-data results

Model	MAE (kWh)	RMSE (kWh)	R-squared

Random Forest	3.317	6.673	0.763
Previous-day baseline	4.869	9.685	0.500

Training rows: 10,792. Test rows: 2,705. Test starts 2025-04-08 09:00:00. These results measure synthetic one-hour-ahead performance, not real-campus or recursive day-ahead performance.



Anomaly alerts and saving scenarios

Isolation Forest is fitted on training data only, with contamination=0.02. Its outlier labels indicate unusual measurements, not confirmed wastage. High-side alerts with positive actual-minus-predicted energy become review candidates. Scenario savings equal flagged positive excess times an assumed recoverable fraction times an editable tariff. Default inputs are 25% and INR 8/kWh, neither verified campus assumptions. Savings cover the holdout period only and exclude fixed charges, demand charges and taxes.

Dashboard and execution

Install requirements, then run: python -m streamlit run app.py. The dashboard provides CSV upload, building filters, observed energy and cost scenario cards, a quality report, EDA, holdout model metrics, actual/predicted charts, alerts, recursive forecasts and downloadable CSV results. The command-line pipeline runs with: python energy.py --output results. See README for virtual environment setup.

Limitations and responsible interpretation

This is a local batch prototype, not a live smart-meter integration. Synthetic data cannot validate actual savings. Missing records can understate totals; building totals are not area-normalized. Forecasts have no calibrated uncertainty bounds. Recursive forecast errors accumulate and require separate multi-step backtesting before deployment. Special holiday dates must be supplied. Operational changes require human review, especially for safety-critical systems. Core processing and sample forecasts were executed; interactive browser behavior was not manually tested here.

Technical documentation

Scikit-learn IsolationForest:
https://scikit-learn.org/1.5/modules/generated/sklearn.ensemble.IsolationForest.html
Streamlit widgets: https://docs.streamlit.io/develop/api-reference/widgets